



Les 5 familles d'architectures RAG

Avec le fil rouge de Léa, ingénieure support — boîte à outils, reranking, évaluation, guide de décision et exercices actifs

01 · Hybrid — matche 🎯

02 · Graph — connecte 🕸

03 · Agentic — raisonne 🧠

04 · Corrective — note 📝

05 · Multimodal — voit 👁

06 · Évaluation — mesure 📏

- **Rencontre avec Léa** — le fil rouge du cours
- **Module 0** — Le socle : c'est quoi, un RAG ?
- **Annexe A** — Boîte à outils (chunking, embedding, top-k...)
- **Module 1** — Hybrid RAG + BM25/RRF chiffré + reranking
- **Module 2** — Graph RAG
- **Module 3** — Agentic RAG
- **Module 4** — Corrective RAG (CRAG) + self-eval vs gradeur
- **Module 5** — Multimodal RAG
- **Module 6** — Évaluer son RAG (golden set, métriques)
- **Guide de décision** — quelle famille choisir
- **Erreurs classiques + Exercice — Trouve l'erreur**
- **Architecture de référence** (production)
- **Corpus fil rouge** — les dossiers de Léa + TP niveaux 1 à 4
- **Synthèse, quiz + Exercice — Schéma ↔ Famille**
- **Glossaire & Références**

v5 — version joie & illustrations · réponses regroupées en fin de document pour un vrai auto-test



Fil rouge

Rencontre avec Léa 🙌*Le personnage qui traverse tout le cours*

Léa est ingénieure support depuis deux ans. Elle gère une base de tickets, un contrat cadre avec plusieurs partenaires, et des rapports internes truffés de graphiques. Elle a branché un chatbot RAG sur ces documents — et déchanté vite : il invente des réponses, rate des références précises, ignore les liens entre ses partenaires, et ne sait pas lire une courbe. Six questions vont jaloner ce cours ; chacune trouve sa réponse dans un module.

Module	La question de Léa
0 — Socle	« Pourquoi mon chatbot invente des réponses au lieu de lire nos docs ? »
1 — Hybrid	« Il ne retrouve jamais le ticket ERR-4042 quand je le demande mot pour mot. »
2 — Graph	« Quelles entreprises partenaires utilisent la techno X sur des projets communs ? »
3 — Agentic	« Peux-tu comparer nos ventes du dernier trimestre (SQL) aux tendances du marché (web), en citant la note interne ? »
4 — Corrective	« Le corpus est bourré de doublons et de vieux tickets obsolètes, comment éviter qu'il réponde n'importe quoi ? »
5 — Multimodal	« Le rapport financier a une courbe de ventes p.12, il ne sait pas la lire. »
6 — Évaluation	« On a testé trois architectures sur nos données, laquelle est vraiment la meilleure ? »



Les travaux pratiques du cours sont littéralement **les dossiers de Léa** : un même corpus fil rouge, repris à chaque niveau de TP.

Module 0



Le socle : c'est quoi, un RAG ?

L'examen à livre ouvert

Objectif : comprendre le principe indexer → récupérer → générer, et pourquoi le RAG naïf a des angles morts.



Léa : « Pourquoi mon chatbot invente des réponses au lieu de lire nos docs ? »

Diagnostic : les 4 faiblesses du RAG naïf

RAG = Retrieval-Augmented Generation (Génération Augmentée par Recherche).



Le RAG « naïf » a 4 faiblesses : il rate les mots-clés exacts, ignore les relations entre documents, fait confiance aveuglément à ce qu'il retrouve, et ne sait pas lire un graphique. **Les 5 architectures suivantes sont 5 remèdes à ces faiblesses.** Le module 6 ajoute un 6e remède : mesurer si le remède fonctionne vraiment. C'est exactement le chatbot que Léa a branché sur ses docs.



Annexe A

Boîte à outils RAG *Les briques utilisées dans tous les modules suivants*

Objectif : donner un sens concret à chunking, embedding, similarité cosinus, base vectorielle et top-k avant de les voir en action.

Terme	Définition concrète
 Chunking	Découpage d'un document en fragments (chunks) de taille fixe (ex. 300-500 tokens) avec un overlap (chevauchement, ex. 50 tokens) entre fragments consécutifs pour ne pas couper une idée à la frontière. Trop petit → perd le contexte ; trop grand → dilue le signal utile dans le top-k.
 Embedding	Fonction qui transforme un texte en vecteur de nombres (ex. 768 dimensions) tel que deux textes de sens proche ont des vecteurs proches dans l'espace. C'est ce qui permet de chercher par « sens » plutôt que par mot exact.
 Similarité cosinus	Mesure de l'angle entre deux vecteurs (1 = identiques en direction, 0 = sans rapport). Sert à classer les documents indexés par proximité de sens avec la question.
 Base vectorielle	Index spécialisé dans la recherche du plus proche voisin sur des millions de vecteurs (FAISS, Chroma, Qdrant, pgvector...). Sans elle, comparer la question à chaque document un par un serait trop lent.
 Top-k	Nombre de fragments les mieux classés effectivement transmis au LLM comme contexte. k trop bas → le LLM manque d'info ; k trop haut → le contexte est noyé et le LLM peut se perdre ou halluciner.

 Ces cinq briques suffisent à construire un RAG naïf. Chaque module qui suit ajoute une brique par-dessus pour combler une faiblesse précise.



Module 01

Hybrid RAG

« Meaning + exact matches » — le sens ET la lettre

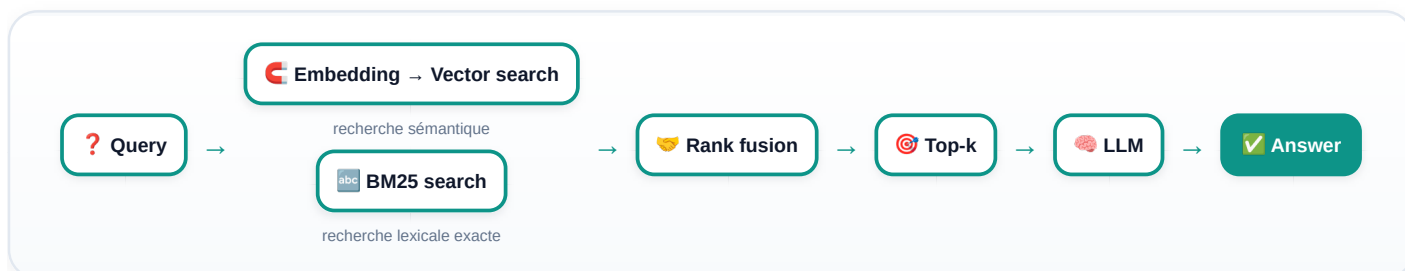


Objectif : savoir pourquoi et comment combiner recherche sémantique et recherche lexicale, puis pourquoi ajouter un reranker derrière.



Léa : « Il ne retrouve jamais le ticket ERR-4042 quand je le demande mot pour mot. »

Répare la faiblesse ① : les mots-clés exacts ratés



Principe

On traduit question + documents en vecteurs pour capturer le **sens** (« connexion lente » ≈ « réseau instable »), tout en gardant une recherche statistique de mots exacts (**BM25**) pour les références, codes, noms propres (ERR-4042, article L.1234-5) que les vecteurs noient — exactement le ticket qui échappe au chatbot de Léa. Les deux classements sont ensuite fusionnés, puis on garde les k meilleurs fragments.

BM25 + Rank Fusion (RRF), chiffré

La fusion la plus utilisée est le **Reciprocal Rank Fusion** : chaque document reçoit un score $1/(k+\text{rang})$ dans chaque liste ($k=60$ par convention), puis on additionne.

Liste vecteurs (par rang) : d1, d2, d3

Liste BM25 (par rang) : d3, d1, d4

$$\text{score}(d1) = 1/(60+1) + 1/(60+2) = 0,01639 + 0,01613 = 0,03252$$

$$\text{score}(d3) = 1/(60+3) + 1/(60+1) = 0,01587 + 0,01639 = 0,03226$$

$$\text{score}(d2) = 1/(60+2) = 0,01613$$

$$\text{score}(d4) = 1/(60+3) = 0,01587$$

Ordre fusionné : d1 > d3 > d2 > d4

Un document présent dans **les deux** listes (d1, d3) bat systématiquement un document présent dans une seule (d2, d4), même s'il n'est pas 1er dans chacune.

Le pattern retrieve → rerank

En production, l'hybrid retrieval seul reste **rapide mais approximatif** (optimisé pour le rappel : ne rien louper). On ajoute un **reranker cross-encoder** : un modèle qui lit la paire (question, document) ensemble — plus lent, mais bien plus précis — et réordonne les candidats avant la coupe top-k finale.



Quand ?

Choix par défaut, surtout corpus techniques/jargonneux avec identifiants

Limite

Ne comprend toujours pas les relations entre documents ; sans reranker, précision plus faible

À retenir

Deux classements valent mieux qu'un ; un reranker derrière vaut mieux que deux

✓ **Checkpoint 1** : un utilisateur tape « erreur POL-2025-114 ». Quelle branche le sauve, et pourquoi un reranker seul (sans BM25) ne suffirait pas forcément ici ?



Module 02

Graph RAG



« Answers in relationships » — la réponse est dans les liens

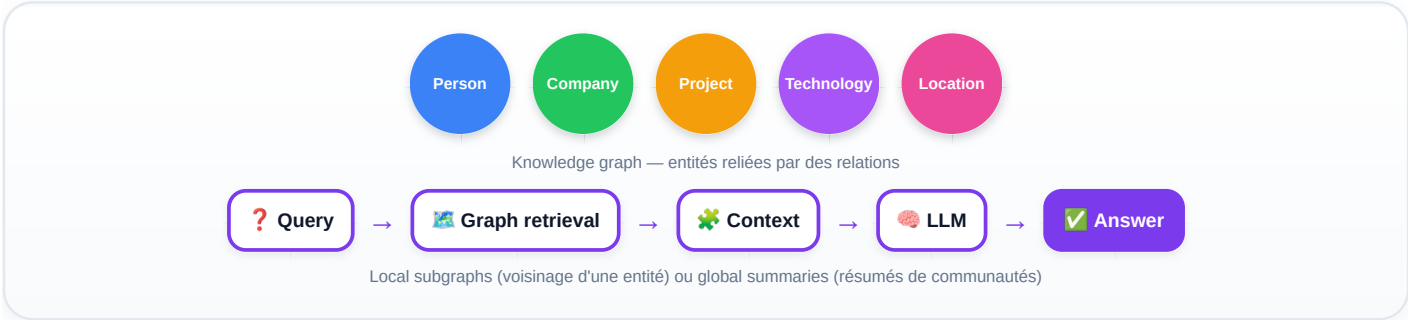


Objectif : reconnaître une question relationnelle multi-sauts et savoir pourquoi le chunk seul ne peut pas y répondre.



Léa : « Quelles entreprises partenaires utilisent la techno X sur des projets communs ? »

 Répare la faiblesse ② : les relations ignorées



Principe

La connaissance n'est plus une pile de chunks isolés : ce sont des **entités reliées par des relations** (Marie → travaille pour → Acme → participe à → Projet H2...), exactement ce qu'il faut pour répondre à Léa. Les **sous-graphes locaux** permettent un raisonnement multi-sauts ; les **résumés globaux** répondent aux questions macro.

Quand ?	Questions relationnelles (qui travaille avec qui, sur quoi, où), traçabilité, synthèses globales
Limite	Construction coûteuse (extraction entités/rerelations) et maintenance du graphe
À retenir	Souvent, aucun chunk ne contient la réponse : elle existe seulement entre les chunks

 **Checkpoint 2** : pourquoi le Hybrid RAG du module 1 galère sur la question de Léa ?



Module 03

Agentic RAG 

« Plan. Retrieve. Verify. » — la récupération devient une stratégie

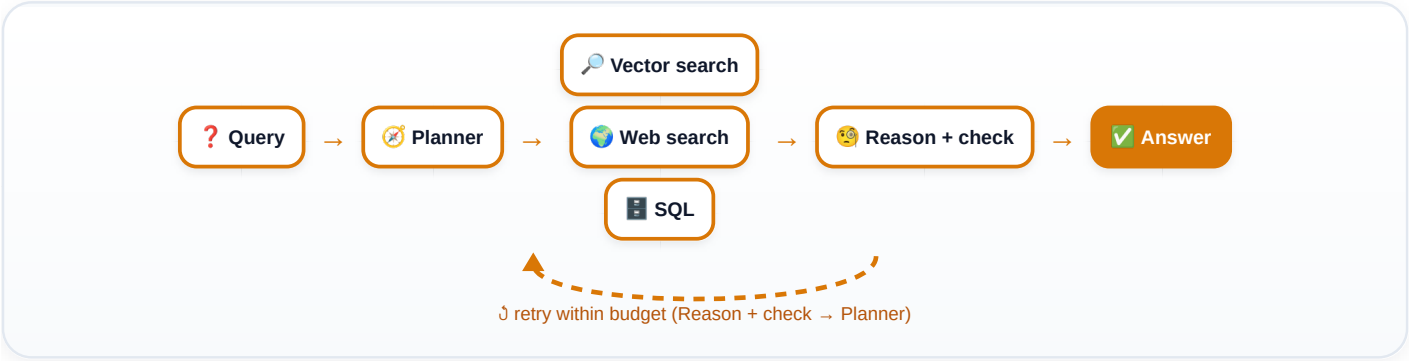


Objectif : comprendre le rôle du planner, de la boucle de vérification, et pourquoi un budget d'itérations est non négociable.



Léa : « Peux-tu comparer nos ventes du dernier trimestre (SQL) aux tendances du marché (web), en citant la note interne ? »

🔧 Répare : la récupération subie → une stratégie qui se vérifie



Principe

Un **agent** décide quoi chercher, où, dans quel ordre : base interne (vecteurs), web, ou base de données (SQL) — les trois sources dont Léa a besoin en une seule question. Après chaque récupération, il **raisonne et vérifie** (*self-evaluation* : il juge son propre raisonnement) ; si c'est insuffisant, il **relance** — mais seulement dans la limite d'un **budget** (nombre d'appels, temps), sinon il répond en l'état.

Quand ?	Questions multi-sources (ventes en SQL + tendances marché en web + note interne en vecteurs)
Limites	Latence et coût élevés, débogage difficile → budget obligatoire sous peine de boucles infinies
À retenir	Ce n'est plus un tuyau, c'est un pilote : planifier, récupérer, vérifier, et savoir s'arrêter



Checkpoint 3 : qu'empêche la boucle « Retry within budget » de tourner indéfiniment ?

Module 04

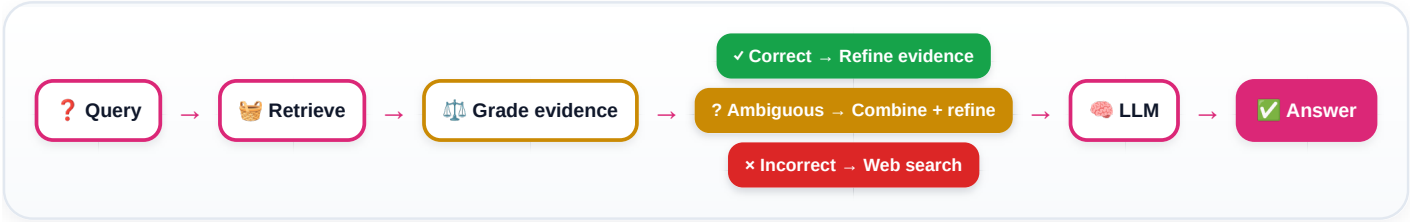
Corrective RAG (CRAG)

« Grade before you trust » — note avant de croire

Objectif : comprendre pourquoi noter la récupération avant génération réduit les hallucinations, et ce que fait chaque verdict.

Léa : « Le corpus est bourré de doublons et de vieux tickets obsolètes, comment éviter qu'il réponde n'importe quoi ? »

Répare la faiblesse ③ : la confiance aveugle



Principe

Un **évaluateur** (scoreur) juge la qualité des **documents retrouvés**, avant même que le LLM génère quoi que ce soit — le filet de sécurité qui manque à Léa contre ses doublons et tickets périmés. Correct → on raffine ; ambigu → on combine avec un complément ; incorrect → on part chercher sur le web avant de combiner.

Quand ?	Corpus bruités/hétérogènes/mouvants, besoin de robustesse et de filet de sécurité
Limites	Un composant de plus à calibrer (le gradeur), latence ajoutée
À retenir	Trois verdicts, trois destins : raffiner, combiner, ou sortir chercher ailleurs

✅ **Checkpoint 4** : documents retrouvés hors-sujet (verdict Incorrect) : que fait CRAG ?

⚠️ **Self-evaluation (module 3) vs Grade evidence (module 4) — ne pas confondre.**
Self-evaluation (Agentic RAG) : l'agent juge *son propre raisonnement ou sa réponse*, après avoir déjà récupéré du contexte — c'est un contrôle a posteriori qui peut déclencher une nouvelle recherche.
Grade evidence (CRAG) : un gradeur juge *la qualité des documents récupérés*, avant toute génération — c'est un filtre a priori sur le contexte, pas sur le raisonnement.
Les deux peuvent coexister dans un même système : un agent qui appelle un pipeline CRAG comme un de ses outils.

✅ **Checkpoint 5** : un agent relit sa propre réponse pour vérifier qu'elle a du sens. Est-ce du self-evaluation façon Agentic RAG, ou du grading façon CRAG ?

Module 05

Multimodal RAG 🧐

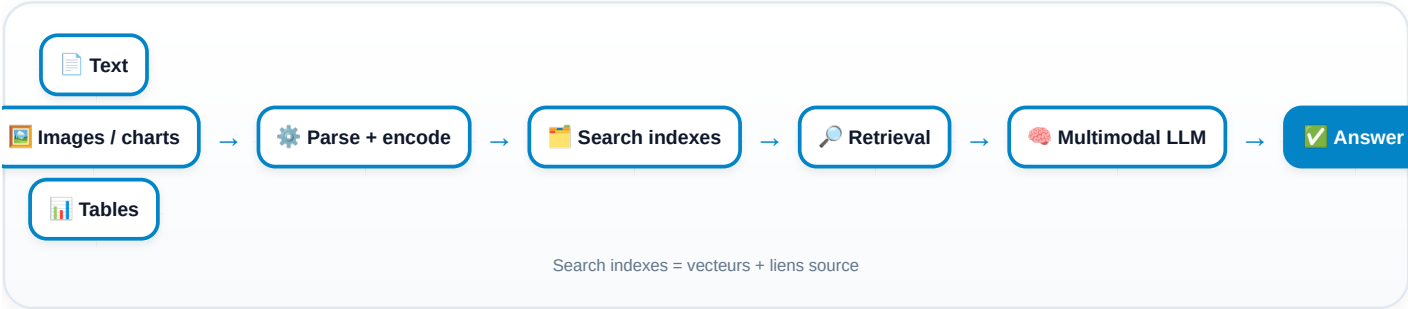
« Go beyond text » — indexe ce que tu lis, y compris ce que tu vois

★

Objectif : savoir quand l'information vit dans une figure/un tableau plutôt que dans le texte, et comment l'indexer.

Léa : « Le rapport financier a une courbe de ventes p.12, il ne sait pas la lire. »

Répare la faiblesse 🕒 : les graphiques illisibles



Principe

L'information vitale dort souvent dans une **courbe**, un **schéma**, un **tableau** — exactement le point mort du chatbot de Léa. Deux voies d'encodage : **embeddings multimodaux** (texte et image dans un même espace vectoriel) ou **captioning** (un modèle décrit l'image, puis indexe le texte). Les **liens source** permettent de citer/afficher l'extrait original.

Quand ?	PDF techniques/financiers, slides, rapports où figures et tables portent le sens
Limites	Qualité des embeddings multimodaux, alignement des modalités, coût
À retenir	Un chunk n'est pas forcément un paragraphe : ça peut être une figure + légende + tableau



Module 06

Évaluer son RAG

Sans mesure, « comparer » n'est qu'une impression



Objectif : construire un golden set et savoir lire les 4 métriques standard avant de déclarer qu'une architecture est « meilleure ».

 **Léa** : « On a testé trois architectures sur nos données, laquelle est vraiment la meilleure ? »

 **Répare** : le « ça marche » au feeling

Le golden set

Un jeu de référence de **questions + réponses attendues + sources attendues**, construit à la main sur ton propre corpus (10 à 30 paires suffisent pour démarrer). C'est la seule façon de comparer deux architectures sur un même étalon plutôt qu'au feeling — la seule vraie réponse à la question de Léa.

Les 4 métriques

Métrique	Ce qu'elle mesure
 Context precision	Parmi les fragments récupérés, quelle proportion est réellement pertinente ? (mesure le bruit)
 Context recall	Parmi les fragments réellement pertinents qui existent dans le corpus, quelle proportion a été récupérée ? (mesure les oublis)
 Faithfulness (fidélité)	La réponse générée s'appuie-t-elle uniquement sur le contexte fourni, ou invente-t-elle des éléments absents ? (mesure l'hallucination)
 Answer relevance	La réponse répond-elle effectivement à la question posée, même si elle est fidèle au contexte ? (mesure le hors-sujet)

 Des frameworks comme **RAGAS** calculent ces 4 métriques automatiquement à partir du golden set, du contexte récupéré et de la réponse générée.

 Un bon context recall avec une mauvaise faithfulness signale un problème de **génération** (le LLM ignore le contexte) ; l'inverse signale un problème de **récupération** (le contexte est incomplet).



Guide

Guide de décision 

Quelle famille choisir, dans quel ordre se poser les questions



Arbre de décision (à parcourir dans l'ordre, premier « oui » gagne)

- 1

Le corpus contient des **références/codes exacts** à retrouver mot pour mot ? → **Hybrid RAG** (+ reranker)
- 2

La question porte sur des **relations entre entités** (qui, avec qui, sur quel projet) ? → **Graph RAG**
- 3

La réponse exige de **croiser plusieurs sources hétérogènes** (SQL + web + interne) ? → **Agentic RAG**
- 4

Le corpus est **bruité, mouvant ou incomplet**, avec risque de mauvais retrieval ? → **Corrective RAG**
- 5

Le sens est porté par des **figures, tableaux, graphiques** ? → **Multimodal RAG**
- 6

Aucun cas ci-dessus ne domine ? → **Hybrid RAG** par défaut, puis instrumenter avec le module 6 avant d'ajouter de la complexité

Coût / latence / complexité / fiabilité

Architecture	Problème ciblé	Complexité	Latence	Fiabilité
 Hybrid RAG	Rappel faible sur termes exacts	Faible	Faible	Moyenne
 Graph RAG	Questions relationnelles multi-hop	Élevée	Moyenne	Moyenne-haute
 Agentic RAG	Sources hétérogènes, raisonnement	Élevée	Haute	Variable (dépend du planner)
 Corrective RAG	Hallucination sur contexte pauvre	Moyenne	Moyenne	Haute
 Multimodal RAG	Contenu non-textuel	Élevée	Moyenne-haute	Moyenne-haute

 Ces patterns se combinent : rien n'empêche un Graph RAG noté par un gradeur CRAG, ou un Agentic RAG dont l'outil « vector search » est lui-même un Hybrid RAG + reranker.



Repères

Erreurs classiques *Ce qui casse un RAG en silence*

- **Chunks trop gros** : chaque fragment mélange plusieurs idées, le vecteur résultant est flou et le retrieval devient imprécis.
- **Overlap nul** : une idée coupée pile à la frontière entre deux chunks devient invisible pour les deux.
- **Top-k trop élevé** : le contexte noie le signal utile dans du bruit, le LLM peut se perdre ou halluciner sur un détail périphérique.
- **Pas de reranking** : bon rappel, mauvaise précision — le LLM reçoit des candidats pertinents mais mal classés.
- **Absence de citations** : impossible de vérifier ou tracer d'où vient une affirmation de la réponse.
- **Gradeur CRAG non calibré** : trop strict, il déclenche des web search inutiles ; trop laxiste, il laisse passer du mauvais contexte.
- **Agent sans budget** : boucles de retry coûteuses, voire infinies, sans garantie de réponse dans un temps raisonnable.

Exercice — Trouve l'erreur

Chaque pipeline ci-dessous contient une erreur de conception. Identifie-la avant de lire les réponses en fin de document.

A. Hybrid RAG : les listes vecteurs et BM25 sont simplement concaténées avant le top-k, sans étape de fusion.

B. Agentic RAG : le Planner peut relancer « Reason + check » tant que le résultat n'est pas jugé satisfaisant, sans limite de nombre d'appels.

C. Corrective RAG : le gradeur classe systématiquement les documents en « Correct », quel que soit leur contenu.

D. RAG naïf : chunks de 100 tokens, top-k = 50.

E. Graph RAG : le graphe de connaissances n'est jamais régénéré après l'ajout de nouveaux documents.



Les modules Graph, Agentic et Multimodal ne remplacent pas ce socle : ils l'enrichissent ou l'orchestrent (un agent peut appeler ce pipeline comme un outil parmi d'autres ; un Graph RAG peut l'utiliser comme récupérateur de secours sur les entités non couvertes par le graphe).



Fil rouge

Les dossiers de Léa

Corpus fil rouge & travaux pratiques



Corpus fil rouge (fictif, ~20 pages)

Le corpus de Léa, réutilisé dans les 4 TP pour que les résultats soient réellement comparables d'une architecture à l'autre :

Composant	Contenu
 Notes de support	Tickets avec identifiants exacts (ex. ERR-4042, POL-2025-114) — pour tester la branche BM25
 Contrat	Articles numérotés et clauses croisées entre entreprises/personnes/projets — pour tester le Graph RAG
 Rapport avec courbes	Quelques pages avec un graphique de ventes trimestrielles et un tableau — pour tester le Multimodal RAG

TP progressifs

- Niveau 1** — RAG naïf sur le corpus de Léa (chunks + embeddings + top-k), puis ajoute BM25 + fusion RRF, puis un reranker. Compare sur 5 questions dont 2 avec références exactes.
- Niveau 2** — construis un mini-graphe (entités/rerelations) à partir du contrat ; compare hybrid vs graph sur 3 questions relationnelles.
- Niveau 3** — ajoute un gradeur LLM devant ta récupération (CRAG) ; teste sur des questions sans réponse dans le corpus et observe le repli web.
- Niveau 4** — construis un golden set de 10 paires question/réponse sur le corpus de Léa ; calcule les 4 métriques du module 6 (à la main ou avec RAGAS) pour chacune des architectures testées aux niveaux 1 à 3 — de quoi enfin répondre à sa question du module 6.



Bilan

Synthèse express

à coller sur ton mur





#	Famille	Devise	Super-pouvoir	Cas type	Vigilance
01	 Hybrid	Sens + exact	Rappel maximal	Jargon, références	Réglage fusion + reranker
02	 Graph	Réponses dans les liens	Multi-sauts + synthèses globales	« Qui avec qui, sur quoi ? »	Coût de construction
03	 Agentic	Plan. Retrieve. Verify.	Multi-sources adaptatif	Questions complexes	Budget, latence
04	 Corrective	Note avant de croire	Robustesse, filet web	Corpus bruités	Calibration du gradeur
05	 Multimodal	Au-delà du texte	Figures & tables interrogables	PDF riches	Embeddings multimodaux
06	 Évaluation	Mesurer avant d'affirmer	Comparaison objective	Choisir/arbitrer entre familles	Golden set représentatif

 **Ces familles se combinent.** Un système production sérieux = hybride + reranker (01) + gradeur (04) + touches multimodales (05), parfois orchestré par un agent (03), et toujours mesuré (06).

Quiz d'auto-évaluation

- Question « contrat DC-2024-091 » : quelle branche de l'Hybrid RAG est décisive ?
- « Résume les grands thèmes de tout le corpus » : quel mode du Graph RAG ?
- Dans l'Agentic RAG, quel mécanisme borne les relances ?
- Dans CRAG, verdict « Incorrect » → quelle action ?
- Répondre sur un graphique d'un PDF : quelle famille ?
- Quel composant du module 1 fusionne les deux classements ?
- Un reranker cross-encoder est-il plus rapide ou plus lent qu'un bi-encoder, et pourquoi l'utiliser quand même ?
- Bon context recall mais mauvaise faithfulness : le problème vient de la récupération ou de la génération ?
- Dans l'arbre de décision, quel est le choix par défaut si aucun critère ne domine ?

Réponses en fin de document, avec les checkpoints et les exercices.



Exercice

Schéma ↔ Famille *Associe chaque flux à son architecture*

Six flux ci-dessous, dans le désordre. Retrouve la famille (RAG naïf, Hybrid, Graph, Agentic, Corrective, Multimodal) derrière chacun avant de consulter les réponses en fin de document.

1. Query → Embedding → Vector search → Top-k → LLM → Answer

2. Query → Planner → (Vector search / Web search / SQL) → Reason + check → (retry within budget) → Answer

3. Query → Graph retrieval → Context → LLM → Answer

4. Text / Images / Tables → Parse + encode → Search indexes → Retrieval → Multimodal LLM → Answer

5. Query → Retrieve → Grade evidence (Correct / Ambiguous / Incorrect) → ... → LLM → Answer

6. Query → (Embedding + Vector search) & (BM25 search) → Rank fusion → Top-k → LLM → Answer



Ressources

Glossaire

15 termes pour ne plus décrocher



RAG

Retrieval-Augmented Generation — génération augmentée par une étape de recherche documentaire.

Chunk

Fragment de document obtenu par découpage (chunking), unité de base indexée et récupérée.

Embedding

Représentation vectorielle d'un texte capturant son sens.

Similarité cosinus

Mesure de proximité entre deux vecteurs, utilisée pour classer les résultats du vector search.

Base vectorielle

Index optimisé pour la recherche de plus proches voisins parmi des embeddings (FAISS, Chroma, Qdrant...).

BM25

Algorithme de recherche lexicale par pertinence statistique des mots exacts.

RRF (Reciprocal Rank Fusion)

Méthode de fusion de plusieurs classements en un seul, basée sur l'inverse du rang.

Reranker / cross-encoder

Modèle qui réordonne un ensemble restreint de candidats en évaluant chaque paire (question, document) conjointement.

Top-k

Nombre de fragments finalement transmis au LLM comme contexte.

Knowledge graph

Représentation de la connaissance sous forme d'entités et de relations entre elles.

Multi-hop

Question dont la réponse nécessite de traverser plusieurs relations/documents successivement.

Agent

Composant qui planifie ses actions (quels outils appeler, dans quel ordre) plutôt que de suivre un pipeline fixe.

Budget (agentic)

Limite de nombre d'itérations ou de temps imposée à un agent pour garantir une réponse finale.

Fidélité (faithfulness)

Degré auquel une réponse générée reste ancrée dans le contexte fourni, sans invention.

Golden set

Jeu de questions/réponses de référence servant d'étalon pour évaluer un système RAG.



Ressources

Références

Pour aller plus loin





Papiers fondateurs

- Lewis et al., 2020 — introduction du RAG (retrieval-augmented generation) pour les tâches génératives à forte intensité de connaissances.
- Edge et al., 2024 — GraphRAG (Microsoft), retrieval augmenté par graphe de connaissances et résumés de communautés.
- Yan et al., 2024 — Corrective RAG (CRAG), évaluation de la qualité du contexte récupéré avant génération.

Outils

- **Frameworks RAG** : LlamaIndex, LangChain, Haystack.
- **Bases vectorielles** : FAISS, Chroma, Qdrant, pgvector.
- **Recherche lexicale** : rank_bm25 (Python).
- **Évaluation** : RAGAS, TruLens, DeepEval.



Bilan

Réponses 

Checkpoints, quiz, exercices — à consulter après avoir répondu



Checkpoints

Checkpoint 1 — BM25 sauve la mise car la référence exacte est rarement proche vectoriellement d'autre chose qu'elle-même. Un reranker seul ne suffit pas s'il ne voit jamais le bon document dans sa liste de candidats initiale : il réordonne, il ne récupère pas.

Checkpoint 2 — le Hybrid RAG cherche des chunks pertinents individuellement ; ici, la réponse dépend de relations entre plusieurs entités (entreprise ↔ techno ↔ projet) qu'aucun chunk isolé ne relie explicitement.

Checkpoint 3 — le budget (nombre d'appels ou de temps maximum), qui force l'agent à répondre en l'état plutôt que de relancer indéfiniment.

Checkpoint 4 — repli sur web search, puis combinaison avec le contexte existant, au lieu d'halluciner sur des documents hors-sujet.

Checkpoint 5 — self-evaluation façon Agentic RAG : l'agent juge sa propre réponse déjà générée. CRAG note les documents en amont, jamais la réponse finale du LLM.

Quiz

1. BM25 (mots exacts).
2. Global summaries.
3. Le budget (retry within budget).
4. Repli sur Web search puis combinaison.
5. Multimodal RAG.
6. Rank fusion.
7. Plus lent (il évalue chaque paire conjointement) ; on l'utilise quand même car il est bien plus précis, sur un petit nombre de candidats déjà pré-filtrés par une recherche rapide.
8. De la génération : le contexte contient l'information nécessaire (bon recall) mais le LLM ne s'y ancre pas correctement (mauvaise fidélité).
9. Hybrid RAG par défaut, puis instrumenter avec le module 6 avant d'ajouter de la complexité.

Exercice — Trouve l'erreur

- A. Pas de fusion pondérée : un document présent dans les deux listes n'est pas favorisé, le classement final peut être pire qu'une seule des deux listes.
- B. Pas de budget : la boucle peut tourner indéfiniment, coût et latence non maîtrisés.
- C. Gradeur non calibré (biaisé vers « Correct ») : il ne joue plus son rôle de filet de sécurité, le corpus bruité passe sans filtre.
- D. Deux erreurs cumulées : chunks de 100 tokens (trop petits : le contexte de chaque idée est perdu, cf. annexe A) et top-k = 50 (beaucoup trop élevé : ~5 000 tokens de contexte noient le LLM au lieu de l'aider).
- E. Graphe jamais mis à jour : les nouvelles entités et relations restent invisibles au retrieval.

Exercice — Schéma ↔ Famille

1. RAG naïf (module 0) — pas de BM25, pas de fusion, pas de reranker.
2. Agentic RAG (module 3).
3. Graph RAG (module 2).
4. Multimodal RAG (module 5).
5. Corrective RAG / CRAG (module 4).
6. Hybrid RAG (module 1).

 **Bravo, tu es arrivé·e au bout !** Tu sais désormais choisir, construire, corriger et mesurer un RAG.
Léa aussi te dit merci : son chatbot tient enfin la route. 

Fait avec  pour réviser en souriant — pour le PDF : imprimer avec « graphiques d'arrière-plan » activé.